Slovenská technická univerzita v Bratislave
Fakulta informatiky a informačných technológií

Protokol k zadaniu

DBS — Databázové technológie

Zadanie 3: Implementácia fyzického modelu a procesov

Reštauračný systém

Študent: Dávid Boldog, Tomáš Bubeník
Študijná skupina: Štvrtok 11:00
Cvičiaci: Ing. Matej Petrík
Akademický rok: 2025/2026
Dátum: 3. mája 2026

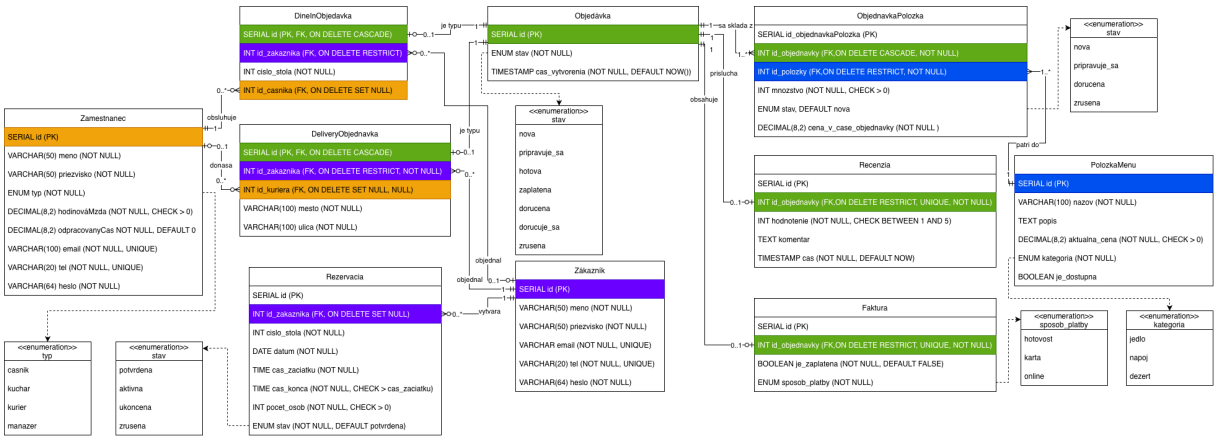
Obsah

2 Zmeny oproti Zadaniu 1

Model ostal tematicky aj vzťahovo rovnaký ako v Zadaní 1: jadrom systému je spoločná tabuľka `Objednavka`, ktorú špecializujú tabuľky `DineInObjednavka` a `DeliveryObjednavka`. Pri implementácii fyzického modelu sme doplnili konkrétne PostgreSQL dátové typy, ENUM hodnoty, referenčné akcie a indexy potrebné pre procesy v tomto zadaní.

Najdôležitejšie spresnenia voči návrhu sú:

- stavové atribúty sú implementované ako PostgreSQL ENUM typy,
- historická cena položky objednávky je uložená v `ObjednavkaPolozka.cena_v_case_objednavky`,
- prekrývanie rezervácií sa nevynucuje cez `CHECK`, pretože ide o medziriadkové pravidlo; procesy ho kontrolujú dotazom,
- proces vytvorenia dine-in objednávky po úspešnom overení prepne rezerváciu do stavu `aktivna`.



Obr. 1: Aktuálny fyzický model databázy

3 Fyzický model - implementácia

Fyzický model je implementovaný v súbore `src/schema.sql`. Skript najprv odstráni existujúce tabuľky a ENUM typy, následne vytvorí všetky typy, tabuľky, integritné obmedzenia a vlastné indexy.

Tabuľka	Účel
`Zamestnanec`	Evidencia čašníkov, kuchárov, kuriérov a manažérov.
`Zakaznik`	Registrovaní zákazníci pre rezervácie, dine-in a donášku.
`PolozkaMenu`	Aktuálna ponuka jedál, nápojov a dezertov.
`Objednavka`	Spoločná entita pre všetky objednávky.
`DineInObjednavka`	Špecializácia objednávky pri stole.
`DeliveryObjednavka`	Špecializácia donáškovej objednávky.
`ObjednavkaPolozka`	M:N vzťah objednávky a menu položiek so snímkou ceny.
`Rezervacia`	Rezervácia konkrétneho stola v časovom intervale.
`Faktura`	Platobný doklad k objednávke.
`Recenzia`	Jedno hodnotenie k jednej objednávke.

Tabuľka 1: Zoznam fyzických tabuliek

Integritné obmedzenia

Model používa primárne kľúče typu `SERIAL`, cudzie kľúče, `NOT NULL`, `UNIQUE`, `CHECK` a `DEFAULT`. Netriviálne pravidlá sú:

- `email` a `tel` sú unikátne pri zákazníkoch aj zamestnancoch,
- ceny, mzdy, množstvá a počet osôb musia byť kladné,
- hodnotenie recenzie je obmedzené na interval 1 až 5,
- koniec rezervácie musí byť po začiatku rezervácie,
- `Faktura.id_objednavky` a `Recenzia.id_objednavky` sú unikátne, teda k objednávke existuje najviac jedna faktúra a najviac jedna recenzia.

ON DELETE stratégie

- `ON DELETE CASCADE` je použité medzi `Objednavka` a jej detailmi, aby sa neosireli špecializačné a položkové záznamy.
- `ON DELETE RESTRICT` chráni historické objednávky pred zmazaním zákazníka alebo položky menu, na ktorú sa odkazujú.
- `ON DELETE SET NULL` sa používa pri voliteľných väzbách na zamestnanca alebo zákazníka, kde chceme zachovať historický záznam aj po odstránení osoby.

4 Seed dáta

Seed dáta sú generované skriptom `seed.py` pomocou knižnice Faker so slovenskou lokalizáciou `sk_SK`. Generovanie je reprodukovateľné, pretože skript používa fixný seed `SEED = 42`. Výstupom je súbor `seed.sql`, ktorý vkladá dáta do všetkých hlavných tabuliek a na konci nastaví hodnoty sekvencií cez `setval`.

Tabuľka	**Počet záznamov**
`Zamestnanec`	25
`Zakaznik`	600
`PolozkaMenu`	50
`Rezervacia`	1 500
`Objednavka`	4 000
`DineInObjednavka`	2 338
`DeliveryObjednavka`	1 662
`ObjednavkaPolozka`	10 764
`Faktura`	3 621
`Recenzia`	1 177
Spolu	**25 737**

Tabuľka 2: Počty záznamov po seede

Distribúcia dát je biznisovo motivovaná: objednávky sú rozdelené približne na 60 % dine-in a 40 % delivery, zákazníci sú vyberaní váženou náhodnosťou, takže časť zákazníkov objednáva často a časť vôbec. Rezervácie sú rozložené medzi minulosť, aktuálny deň a budúcnosť, pričom budúce rezervácie neobsahujú dokončené stavy.

Konzistencia je zabezpečená poradím generovania dát. Najprv vznikajú referenčné entity (`Zakaznik`, `Zamestnanec`, `PolozkaMenu`), potom rezervácie, objednávky, špecializácie objednávok, položky, faktúry a recenzie. Generátor vyberá iba existujúce cudzie kľúče a rešpektuje roly zamestnancov pri čašníkoch a kuriéroch.

5 Proces 1 - Vytvorenie dine-in objednávky

Popis procesu

Proces pokrýva príchod zákazníka s rezerváciou a vytvorenie objednávky pri stole. Vstupom sú číslo stola, zákazník, dátum a čas príchodu, čašník a tri objednávané položky s množstvami. Výstupom je detail vytvorenej objednávky alebo zoznam validačných chýb.

Proces overuje existenciu zákazníka, potvrdenú rezerváciu v danom časovom okne, rolu čašníka a dostupnosť všetkých položiek menu. Ak všetky kontroly prejdú, v jednom SQL príkaze sa aktivuje rezervácia a vložia sa záznamy do `Objednavka`, `DineInObjednavka` a `ObjednavkaPolozka`.

SQL implementácia

Implementácia je v súbore `src/queries.sql`. Používa CTE, `UPDATE`, `INSERT ... RETURNING` a finálny `SELECT`. Nepoužíva PL/pgSQL blok `DO`, vlastné uložené funkcie, procedúry, triggery ani kurzory.

```sql
WITH
params AS (...),
input_items(polozka_id, mnozstvo) AS (...),
valid_reservation AS (...),
valid_items AS (...),
validation_errors AS (...),
activated_reservation AS (
    UPDATE Rezervacia r
    SET stav = 'aktivna'
    FROM valid_reservation vr
    WHERE r.id = vr.id
      AND NOT EXISTS (SELECT 1 FROM validation_errors)
    RETURNING r.id AS id_rezervacie
),
inserted_order AS (
    INSERT INTO Objednavka (stav, cas_vytvorenia)
    SELECT 'nova'::stav_objednavky, p.p_datum + p.p_cas
    FROM params p
    WHERE EXISTS (SELECT 1 FROM activated_reservation)
    RETURNING id, stav, cas_vytvorenia
)
SELECT ...
```

Ukážka spustenia

Testovacie parametre v `queries.sql`:

- stôl: 8,
- zákazník: `id = 320`,
- čas príchodu: `2026-05-03 13:15`,
- čašník: `id = 13`,
- položky: `12 x 1`, `17 x 2`, `24 x 2`.

Položka	Stav	Množstvo	Subtotal
Pizza Margherita	nova	1	13.30
Kurací burger	nova	2	31.82
Cappuccino	nova	2	7.64
Spolu			**52.76**

Tabuľka 3: Očakávaný výstup procesu 1 nad seed dátami

Zdôvodnenie korektnosti

Objednávka sa vloží iba vtedy, keď CTE `validation_errors` nevráti žiadny riadok. Všetky zápisy sú v jednom SQL príkaze, takže nevznikne situácia, že sa vytvorí iba časť objednávky. Snímka ceny sa berie z aktuálnej ceny v `PolozkaMenu` v momente vytvorenia objednávky.

Indexy podporujúce proces

- `idx_rezervacia_stol_datum` podporuje hľadanie rezervácie podľa stola, dátumu a stavu.
- `idx_polozka_dostupna` podporuje filtrovanie dostupných položiek menu.
- `idx_objednavkapolozka_objednavka` zrýchľuje načítanie položiek vytvorenej objednávky.

6 Proces 2 - Výkonnostný report čašníkov

Popis procesu

Manažér chce za zvolený rok vidieť výkon jednotlivých čašníkov. Report pracuje iba so zaplatenými dine-in objednávkami, teda so záznamami, ku ktorým existuje faktúra s `je_zaplatena = TRUE`. Výstup obsahuje počet objednávok, celkovú tržbu, priemernú hodnotu objednávky, priemerné hodnotenie, poradie podľa tržby, percentuálny podiel a kumulatívnu tržbu.

SQL implementácia

Proces používa pohľad `v_dine_in_trzby`, CTE, agregácie a window funkcie. Rok sa filtruje cez rozsah dátumov, nie cez `EXTRACT(YEAR ...)`, aby databáza vedela efektívnejšie použiť index na `Objednavka.cas_vytvorenia`.

```sql
CREATE OR REPLACE VIEW v_dine_in_trzby AS
SELECT o.id, o.cas_vytvorenia, d.id_casnika,
       SUM(op.mnozstvo * op.cena_v_case_objednavky) AS trzba_objednavky
FROM Objednavka o
JOIN Faktura f ON f.id_objednavky = o.id AND f.je_zaplatena = TRUE
JOIN DineInObjednavka d ON d.id = o.id
JOIN ObjednavkaPolozka op ON op.id_objednavky = o.id
WHERE o.stav <> 'zrusena'
GROUP BY o.id, o.cas_vytvorenia, d.id_casnika;

WITH year_bounds AS (
    SELECT '2026-01-01'::TIMESTAMP AS rok_od,
           '2027-01-01'::TIMESTAMP AS rok_do
)
SELECT ...
```

Ukážka výstupu

Poradie	Časník	Obj.	Tržba	Priemer	Hodn.
1	Kristián Kučerová	47	3001.85	63.87	4.13
2	Ľubomíra Miklošová	46	2797.38	60.81	4.00
3	Metod Huba	38	2528.98	66.55	3.75
4	Leonard Macháčeková	34	2460.25	72.36	3.75
5	Irena Janečková	35	2435.39	69.58	3.93

Tabuľka 4: Prvých päť riadkov reportu pre rok 2026

Zdôvodnenie korektnosti

Tržba vzniká ako súčet `mnozstvo * cena_v_case_objednavky`, teda používa historickú cenu uloženú v objednávke, nie aktuálnu cenu menu. Zrušené objednávky a zrušené položky sa do reportu nezapočítavajú. Join na `Faktura` zabezpečí, že report reprezentuje reálne zaplatené tržby.

Indexy podporujúce proces

- `idx_objednavka_cas_vytvorenia` podporuje rozsahový filter zvoleného roka.
- `idx_faktura_zaplatena` podporuje výber zaplatených faktúr a join na objednávku.
- `idx_objednavkapolozka_objednavka` podporuje agregáciu položiek podľa objednávky.

7 Výkonnosť a indexy

PostgreSQL automaticky vytvára indexy pre `PRIMARY KEY` a `UNIQUE` obmedzenia. Tieto indexy sa do požadovaného minima nepočítajú. V schéme preto vytvárame päť vlastných indexov, ktoré priamo podporujú dva implementované procesy.

`idx_rezervacia_stol_datum`

```
CREATE INDEX idx_rezervacia_stol_datum
    ON Rezervacia (cislo_stola, datum, stav);
```

Index podporuje proces 1, konkrétne validáciu rezervácie pred vytvorením dine-in objednávky. Dopyt filtruje tabuľku `Rezervacia` podľa `cislo_stola`, `datum` a `stav`. Bez indexu musí databáza prejsť všetky rezervácie sekvenčným skenom a až potom odfiltrovať konkrétny stôl, dátum a stav. Index má odstrániť práve tento sekvenčný sken nad tabuľkou `Rezervacia`; zvyšné podmienky na zákazníka a časový interval sa potom aplikujú iba na malú množinu kandidátov.

`idx_polozka_dostupna`

```
CREATE INDEX idx_polozka_dostupna
    ON PolozkaMenu (je_dostupna);
```

Index podporuje proces 1 pri validácii, že objednávané položky menu existujú a sú dostupné. Dopyt používa podmienku `je_dostupna = TRUE`. Cieľom indexu je obmedziť skenovanie nedostupných položiek pri zostavovaní dostupného menu alebo pri validácii vstupu objednávky. Keďže tabuľka menu je v našom seede malá, optimalizátor môže aj tak zvoliť sekvenčný sken; index je však opodstatnený pre prípad väčšieho menu alebo častého filtrovania dostupných položiek.

idx_objednavkapolozka_objednavka

```
CREATE INDEX idx_objednavkapolozka_objednavka
    ON ObjednavkaPolozka (id_objednavky);
```

Index podporuje oba procesy. V procese 1 sa používa pri načítaní detailu novo vytvorenej objednávky a v procese 2 pri agregácii tržby objednávky. Dopyty často hľadajú všetky riadky `ObjednavkaPolozka` pre jedno konkrétne `id_objednavky`. PostgreSQL nevytvára index automaticky na stĺpci cudzieho kľúča, preto je tento index vlastný. Má obmedziť sekvenčný sken celej tabuľky položiek objednávok a nahradiť ho indexovým vyhľadaním položiek konkrétnej objednávky.

idx_objednavka_cas_vytvorenia

```
CREATE INDEX idx_objednavka_cas_vytvorenia
    ON Objednavka (cas_vytvorenia);
```

Index podporuje proces 2, kde manažérsky report filtruje objednávky za zvolený rok. Dopyt je zámerne zapísaný ako rozsah:

```
WHERE o.cas_vytvorenia >= y.rok_od
  AND o.cas_vytvorenia < y.rok_do
```

Takýto zápis je vhodný pre B-tree index nad `cas_vytvorenia`. Index má obmedziť sekvenčné skenovanie všetkých objednávok pri ročných reportoch. Keby bol filter zapísaný ako `EXTRACT(YEAR FROM cas_vytvorenia)`, databáza by musela vypočítať výraz pre každý riadok a tento index by sa používal horšie alebo vôbec.

idx_faktura_zaplatena

```
CREATE INDEX idx_faktura_zaplatena
    ON Faktura (je_zaplatena, id_objednavky);
```

Index podporuje proces 2. Report má počítať iba reálne tržby, preto pohľad `v_dine_in_trzby` joinuje tabuľku `Faktura` a filtruje `je_zaplatena = TRUE`. Prvý stĺpec indexu podporuje filter zaplatených faktúr, druhý stĺpec podporuje následný join na `Objednavka.id`. Index má obmedziť sekvenčný sken tabuľky `Faktura` pri hľadaní zaplatených objednávok.

EXPLAIN porovnanie

Porovnanie sme spravili pre proces 1, konkrétne pre validáciu rezervácie. Test bol spustený nad naplnenou databázou so seed dátami. Najprv sme index `idx_rezervacia_stol_datum` odstránili a spustili plán:

```
DROP INDEX IF EXISTS idx_rezervacia_stol_datum;

EXPLAIN (ANALYZE, BUFFERS)
SELECT r.id, r.id_zakaznika, r.cislo_stola, r.datum,
       r.cas_zaciatku, r.cas_konca, r.stav
FROM Rezervacia r
WHERE r.cislo_stola = 8
  AND r.id_zakaznika = 320
  AND r.datum = DATE '2026-05-03'
  AND TIME '13:15' BETWEEN r.cas_zaciatku AND r.cas_konca
  AND r.stav = 'potvrdena'
ORDER BY r.id
LIMIT 1;
```

Výrez plánu bez indexu:

```
Limit (actual time=0.164..0.164 rows=1 loops=1)
  Buffers: shared hit=16
  -> Sort
        Sort Key: id
        -> Seq Scan on rezervacia r
              Rows Removed by Filter: 1499
              Buffers: shared hit=13
Planning Time: 0.531 ms
Execution Time: 0.189 ms
```

Bez indexu databáza použila `Seq Scan on rezervacia`. To znamená, že prešla všetkých 1 500 rezervácií a 1 499 riadkov zahodila filtrom.

Následne sme index vytvorili znova a spustili ten istý dopyt:

```
CREATE INDEX idx_rezervacia_stol_datum
    ON Rezervacia (cislo_stola, datum, stav);

ANALYZE Rezervacia;

EXPLAIN (ANALYZE, BUFFERS)
SELECT r.id, r.id_zakaznika, r.cislo_stola, r.datum,
       r.cas_zaciatku, r.cas_konca, r.stav
FROM Rezervacia r
WHERE r.cislo_stola = 8
  AND r.id_zakaznika = 320
  AND r.datum = DATE '2026-05-03'
  AND TIME '13:15' BETWEEN r.cas_zaciatku AND r.cas_konca
  AND r.stav = 'potvrdena'
ORDER BY r.id
LIMIT 1;
```

Výrez plánu s indexom:

```
Limit (actual time=0.037..0.037 rows=1 loops=1)
  Buffers: shared hit=1 read=2
  -> Sort
        Sort Key: id
        -> Index Scan using idx_rezervacia_stol_datum on rezervacia r
              Index Cond: ((cislo_stola = 8)
                          AND (datum = '2026-05-03'::date)
                          AND (stav = 'potvrdena'::stav_rezervacie))
              Filter: (id_zakaznika = 320 AND ...)
Planning Time: 0.264 ms
Execution Time: 0.059 ms
```

Po vytvorení indexu sa plán zmenil zo sekvenčného skenu na `Index Scan`. Databáza už nehľadala rezerváciu v celej tabuľke, ale použila indexovú podmienku nad stĺpcami `cislo_stola`, `datum` a `stav`. Aj na relatívne malom seede klesol čas vykonania z približne `0.189 ms` na `0.059 ms`. Zostalo iba malé triedenie podľa `id`, pretože `id` nie je súčasťou indexu; triedi sa však už len veľmi malá množina kandidátnych riadkov.

8 Zhodnotenie

Najnáročnejšou časťou bolo zosúladiť fyzický model s procesmi tak, aby procesy neboli iba jednoduché SELECT dotazy, ale pokrývali reálnu prevádzkovú logiku. Pri procese 1 bolo dôležité

vytvoriť objednávku čisto v SQL bez PL/pgSQL, ale zároveň zachovať validačné kroky a atomickosť zápisu.

Pri implementácii sa ukázalo, že niektoré biznis pravidlá nie je vhodné riešiť obyčajným `CHECK` obmedzením. Typickým príkladom je prekrývanie rezervácií, ktoré vyžaduje kontrolu voči iným riadkom. Toto pravidlo preto riešime validačným dotazom v procese.

Indexy sme nenavrhovali plošne na všetky stĺpce. Zamerali sme sa na časti dopytov, ktoré sa opakujú v procesoch: hľadanie rezervácie podľa stola a dátumu, načítanie položiek objednávky, filter zaplatených faktúr a rozsahový filter objednávok podľa dátumu.

AI nástroje sme použili ako pomoc pri kontrole konzistentnosti textu, formulácii reportu a prepisovaní SQL procesu do čistej deklaratívnej podoby. Výsledné SQL a model sme skontrolovali voči zadaniu a upravili podľa reálnych tabuliek v projekte.